

F01 — Vehicle Presets

Implementation report, and what it means for F09 Project Persistence

Branch ui-vehicle-presets · commits 7b342ee, 5d04506, dff8542 · 15 September 2026

F01 delivers a reusable vehicle preset library: a prefab system where each preset holds a vehicle's technical and economic attributes and chooses the 3D model its instances render with. This report covers how it works, every file operation it performs, and the concrete work required before F09 can persist it.

1. What shipped

Commit	Scope	Contents
7b342ee	Vehicle preset library and scene linking	Domain types, store, panel, presetId on scene objects, scene document v2→v3, shared TextControl
5d04506	Seed presets moved from code to data	defaults.json plus the loader and the test that guards it
dff8542	Dev-only write-back DEV	Vite serve-mode plugin, POST /__save-presets, Save button with confirmation

Files

File	Lines	Responsibility
src/vehicles/types.ts	78	Domain type and validation. Zero imports — no React, Three.js or Zustand
src/vehicles/defaults.json	65	Five seed presets, as data
src/vehicles/defaults.ts	26	Reads and validates the seed file. The single load seam
src/state/presetStore.ts	155	Zustand store: CRUD, import/export, edit baseline, geometry and name resolution
src/components/VehiclePresets.tsx	186	The sidebar panel
vite/savePresets.ts	93	Dev-only write-back endpoint
Tests (types, presetStore, catalog, App)	159	34 tests total in the client suite, 33 passing

One pre-existing failure remains. `models/van.test.ts:19` asserts the van roof is above 2.3 m; commit `ed16f58` remodelled it to 2.24 m without updating the test. Unrelated to F01, deliberately left for the pull request.

2. How vehicle presets work

The prefab/instance split

The central idea is that *what kind of vehicle this is* and *which vehicle is parked here* are separate records:

Record	Represents	Example
VehiclePreset	A vehicle type — the prefab asset. Project-level, shared	"Electric Delivery Van · 22 kWh/100 km · 45,000"
SceneObject	One placed instance in the 3D world	That particular van, parked at x=3, z=7

A scene object stores **no vehicle data at all** — only a transform, an appearance override, and a presetId. Anything needing a vehicle's cost or energy use follows that id to the preset and reads it live. There is no copy to keep in sync, which is why editing a preset updates every placed instance with no propagation step.

Geometry resolution

```
SceneObject.presetId → preset.modelId → objectDefinitions[...] → THREE.Group
└─ unset, or preset deleted → SceneObject.definitionId (fallback)
```

A preset never contains geometry. It holds modelId, a string key into the existing object catalog, satisfying the rule in **AGENTS/typed-objects-plan.md** that a procedural model is presentation, not domain identity. Resolution lives in two hooks in **presetStore.ts** — useResolvedModelId and useResolvedName — each returning a string so Zustand's reference equality prevents needless re-renders.

Deleting a preset therefore **orphans** its instances rather than deleting them: they fall back to definitionId and the name they were placed with, and keep rendering.

The preset record

```
type VehiclePreset = {
  id, name, category, propulsion, // identity
  modelId, // → object catalog key (geometry)
  litresPer100Km, kWhPer100Km, // energy
  batteryCapacityKWh, chargingPowerKW,
  purchaseCost, // economics
};
```

Field names deliberately match the vocabulary already fixed in **docs/tech/simulation.md**, so the scenario engine can consume these records without a renaming pass. The form is grouped into Identity / Energy / Economics / Appearance sections so further attributes from **contracts.md** — range, maintenance, residual values, physical dimensions — drop into an existing section rather than forcing a redesign.

Validation

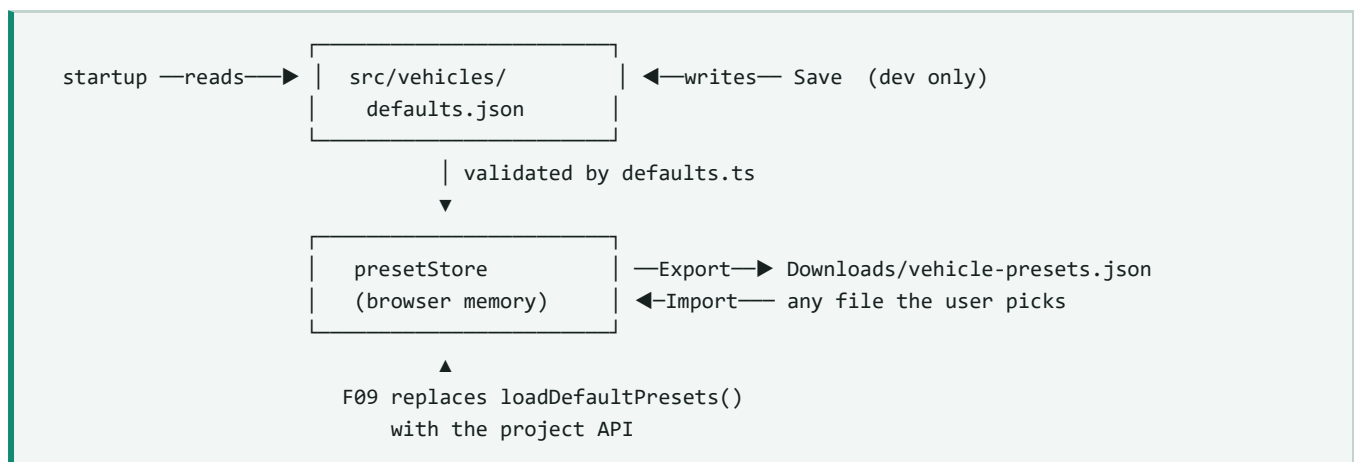
normalizePreset(value, knownModelIds?) is the single gate. It rejects non-finite and negative numbers, unknown propulsion values, empty or over-length names, prototype-polluting ids, and optionally any modelId absent from the catalog. The catalog keys are passed in as an argument rather than imported, which is what keeps the module free of Three.js.

Every write runs through it. updatePreset builds the patched record, validates the *whole* result, and rejects the edit outright if invalid — so a bad draft can never replace valid inputs, as **contracts.md** requires. Ids are immutable: a patch cannot repoint a record.

3. File operations

Four operations move preset data today. Only one writes into the repository.

Operation	Source	Destination	Trigger
Startup load	<code>src/vehicles/defaults.json</code>	Preset store (memory)	Page load, via <code>loadDefaultPresets()</code>
Export	Preset store	<code>Downloads/vehicle-presets.json</code>	Export button
Import	Any file the user picks	Preset store	Import button
Save DEV	Preset store	<code>src/vehicles/defaults.json</code>	Save button → confirm



One file format everywhere

The seed file, the Export download and the write-back request body are all the same shape:

```
{ "version": 1, "presets": [ ... ] }
```

So an exported library can be dropped in as the seed unchanged, and the Save button simply posts `exportPresets()`. The version is checked explicitly on every read, matching `contracts.md`'s requirement that unknown document versions are rejected rather than guessed at.

Import semantics

Import takes a whole library and **replaces** the current one — it does not merge. Every record is validated into a new array first; if any record is invalid, or repeats an id, or the version is unknown, **nothing is imported** and the error names the offending record. There is no partial-import state.

The dev-only write-back

Browsers cannot write to disk, so the page posts to the Vite dev server, which is a Node process that can. The plugin is registered with `apply: "serve"`, so it is absent from production builds entirely — confirmed by grepping the built bundle for the route.

The purity constraint paid for itself here. Because `vehicles/types.ts` imports nothing, the Vite plugin imports the *same* `normalizePreset` the browser uses. The seed file cannot be overwritten with data the application would refuse to load, without duplicating a line of validation logic. The same property is what will let the Fastify server validate identically in F09.

It writes a tracked file. Unlike Export, this lands in the git working tree, so a stray click can appear in a commit. Save therefore asks for confirmation and names the file. Reviewing `git diff` on `defaults.json` before committing is now a sensible habit — that is exactly how an accidental extra preset was caught during development.

4. What this means for F09

F09 must add the PostgreSQL schema, Drizzle definitions and migrations, and implement `/api/v1/projects` CRUD. Today the server has only `GET /health` and `GET /api`, and `db/schema.ts` is an empty placeholder.

What F09 gets for free

- **A single load seam.** `loadDefaultPresets()` is the only place preset data enters the application. F09 replaces its body; no component, store action or test needs to change to point at the API.
- **Fully serializable records.** Presets reference geometry by string id and never hold a `THREE.Group`, so a preset round-trips through JSON and into `jsonb` with no transformation.
- **A validator the server can reuse.** `vehicles/types.ts` is framework-free, so the Fastify route can run the identical checks. `architecture.md` requires exactly this — "shared domain definitions keep the browser and API consistent" — and the write-back plugin already proves it works outside the browser.
- **Versioned documents from day one**, and atomic all-or-nothing load semantics that match the failure behaviour `contracts.md` specifies for saves.

What has to change

Issue	Today	Needed for F09
Loading is synchronous	<code>presets: loadDefaultPresets()</code> runs when the store module is created	An API call is asynchronous. Needs an initial empty state, an explicit load action, and loading/error status. This is the main structural change.
Presets are standalone	A <code>PresetFile</code> of its own	<code>contracts.md</code> nests presets inside one project document alongside analysis settings, the fleet and scenarios. <code>PresetFile</code> becomes a slice of <code>ProjectDocument</code>
No revision concept	No project id, no revision	PUT and DELETE take an expected revision and return 409 on conflict. The client must hold and send it
Validator lives in the client	<code>apps/client/src/vehicles/types.ts</code>	<code>apps/server</code> cannot import across workspace packages today. Needs extraction to a shared package, or the architecture's shared-validation rule is broken by duplication
Scene document is separate	<code>SceneDocument v3</code> lives only in memory	Scenario layouts persist inside the same project document; both version numbers must be handled on read
A second write path	Save writes <code>defaults.json</code>	Decide whether it stays a designer tool or retires once presets come from the database

5. Before starting F09

1. **F03 — Project and Scenario Controls must land first.** This is the stated dependency in the F09 specification, and it is not started: there is no project or scenario component in `src/components/`. F03 defines the project/scenario shape the API has to serve; building the persistence layer first risks persisting the wrong structure.
2. **Extract the domain module to a shared workspace package.** Move `vehicles/types.ts` (and future fleet, charger and analysis types) out of `apps/client` into something both applications depend on — for example `packages/domain`, added to `pnpm-workspace.yaml`. Its zero-import discipline means it moves unchanged. Do this **before** writing routes, or the server will end up with a second copy of the validation rules.
3. **Define and version the project document schema.** Presets, fleet, analysis settings and scenarios in one validated object, with its own schema version distinct from the preset file version and the scene document version.
4. **Make preset loading asynchronous.** Convert the store to start empty and load explicitly, so the API swap is a change of source rather than a change of shape.
5. **Add project identity and revision to client state**, so conflict handling can be implemented rather than retrofitted.
6. **Decide the fate of the write-back.** Keeping it is defensible as a seed-authoring tool; the decision just should not be made by accident.

Suggested order. Steps 2 and 3 are worth doing even before F03 finishes, because they are independent of the UI and unblock server work. Steps 4 and 5 depend on F03's project model. Step 1 gates the routes themselves.

6. Known limitations

- **Preset edits are not undoable.** `HistoryControls` is bound to the scene store, and presets are project data with a different lifetime. The panel has an edit baseline so Escape reverts a field, but Ctrl+Z does not cover presets. Project-level history is F03's decision.
- **The 3D model selector has one option.** Only van is registered in the catalog. The selector is real and the resolution path is tested, but live geometry switching cannot be demonstrated visually until a second model exists.
- **Physical dimensions are not yet a preset field**, so depot bay sizing and space feasibility cannot use them. `contracts.md` already reserves the field.
- **An instance's stored definitionId is captured at placement** and not re-synced if the preset later changes model. It only matters after the preset is deleted, when the instance falls back to the geometry it was placed with. A deliberate, documented trade-off.